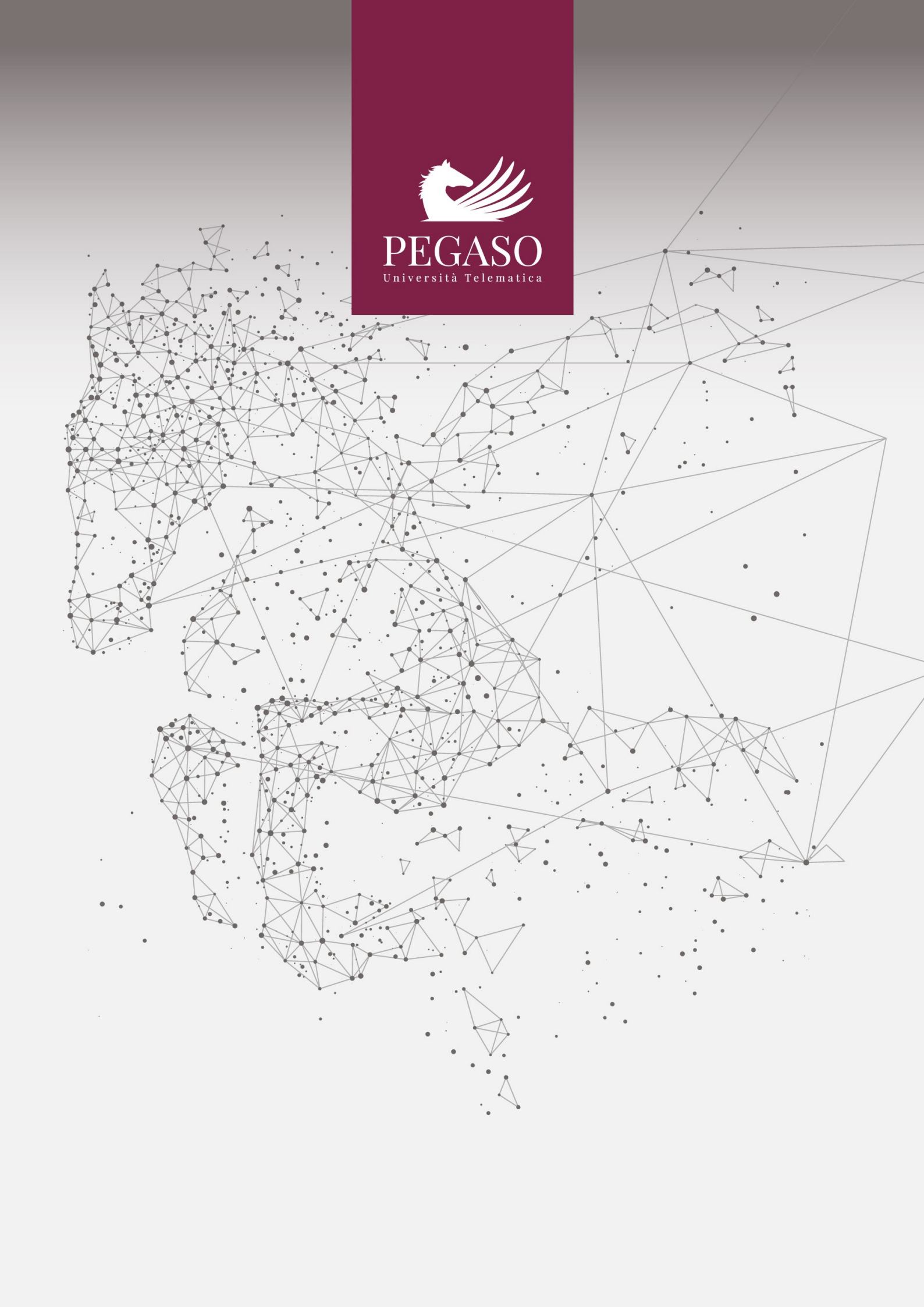

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. RICHIAMO DELLE NOZIONI RELATIVE ALLA GESTIONE DEGLI ARRAY 3

2. ESERCIZI SULLA GESTIONE DEGLI ARRAY IN ASSEMBLY MIPS 7

3. RICHIAMO DELLE NOZIONI RELATIVE ALLA GESTIONE DELLE STRINGHE 9

4. ESERCIZI SULLA GESTIONE DELLE STRINGHE IN ASSEMBLY MIPS 11

BIBLIOGRAFIA 14

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Richiamo delle nozioni relative alla gestione degli Array

La struttura dati Array è una lista di elementi in cui ogni elemento è direttamente accessibile tramite un *indice* ad esso associato.

Gli elementi di un Array sono tutti dello stesso Tipo e la *dimensione* dell'Array stabilisce il numero degli elementi che lo costituiscono. L'indice varia da 0 al valore della *dimensione* meno 1.

I Linguaggi ad Alto Livello che supportano il Tipo di dati Array mettono a disposizione gli operatori necessari per la gestione degli elementi, in particolare, per la lettura e la scrittura in Memoria di un elemento dell'Array specificato dal relativo indice.

Nei **Linguaggi C, C++ e Java** l'operatore che consente l'accesso allo specifico elemento di un Array è denotato dalle parentesi quadre, quindi le semplici istruzioni di assegnamento:

val = A[k] *lettura* dell'elemento A[k] in Memoria

A[k] = val *scrittura* dell'elemento A[k] in Memoria

consento, rispettivamente, l'accesso in Memoria in lettura ed in scrittura per la gestione diretta dell'elemento dell'Array il cui indice è dato dal valore della variabile k.

Il **Linguaggio Assembly** NON DISPONE di operatori come le parentesi quadre per gestire l'accesso in Memoria per riferimento mediante l'indice. Occorre fornire l'indirizzo di Memoria relativo all'elemento, sia nel caso che il valore dell'indice sia dato esplicitamente mediante una costante numerica, sia nel caso che sia fornito dal valore di una variabile.

La struttura dati Array è una lista di elementi in cui ogni elemento è direttamente accessibile tramite un *indice* ad esso associato.

Gli elementi di un Array sono tutti dello stesso Tipo e la *dimensione* dell'Array stabilisce il numero degli elementi che lo costituiscono. L'indice varia da 0 al valore della *dimensione* meno 1.

I Linguaggi ad Alto Livello che supportano il Tipo di dati Array mettono a disposizione gli operatori necessari per la gestione degli elementi, in particolare, per la lettura e la scrittura in Memoria di un elemento dell'Array specificato dal relativo indice.

Nei **Linguaggi C, C++ e Java** l'operatore che consente l'accesso allo specifico elemento di un Array è denotato dalle parentesi quadre, quindi le semplici istruzioni di assegnamento:

val = A[k] *lettura* dell'elemento A[k] in Memoria

A[k] = val *scrittura* dell'elemento A[k] in Memoria

consento, rispettivamente, l'accesso in Memoria in lettura ed in scrittura per la gestione diretta dell'elemento dell'Array il cui indice è dato dal valore della variabile k.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il **Linguaggio Assembly** NON DISPONE di operatori come le parentesi quadre per gestire l'accesso in Memoria per riferimento mediante l'indice. Occorre fornire l'indirizzo di Memoria relativo all'elemento, sia nel caso che il valore dell'indice sia dato esplicitamente mediante una costante numerica, sia nel caso che sia fornito dal valore di una variabile.

Nel caso delle istruzioni *lh, lhu, lb, lbu* le locazioni consecutive lette sono 2 (2 Byte) oppure una (un Byte). Quindi la scrittura della sequenza nel registro lascia libere delle posizioni, ed è richiesto un COMPLETAMENTO per ottenere una rappresentazione equivalente del dato con 32 bit, che si effettua con due diverse modalità:

- *lh* e *lb* per numeri interi effettuano l'***estensione del segno*** (copiando il valore del bit di segno),
- *lhu* e *lbu* per numeri *unsigned* aggiungono **bit uguali a 0**.

L'Offset ed il Registro Base vengono utilizzati per il calcolo dell'INDIRIZZO DI MEMORIA del BYTE più significativo da cui inizia la sequenza del dato da trasferire (approccio ***Big endian***).

La Modalità di Indirizzamento utilizzata per il calcolo di tale indirizzo è detta ***Indirizzamento tramite Base e Offset***.

INDIRIZZAMENTO TRAMITE BASE E OFFSET

Indirizzo di Memoria =

contenuto del *Registro Base* + *Offset* esteso di segno a 32 bit

Poiché la GESTIONE DI UN ARRAY prevede accessi in Memoria in lettura e scrittura, nella **traduzione in Assembly** sono coinvolte le istruzioni di trasferimento dati *Load* e *Store*.

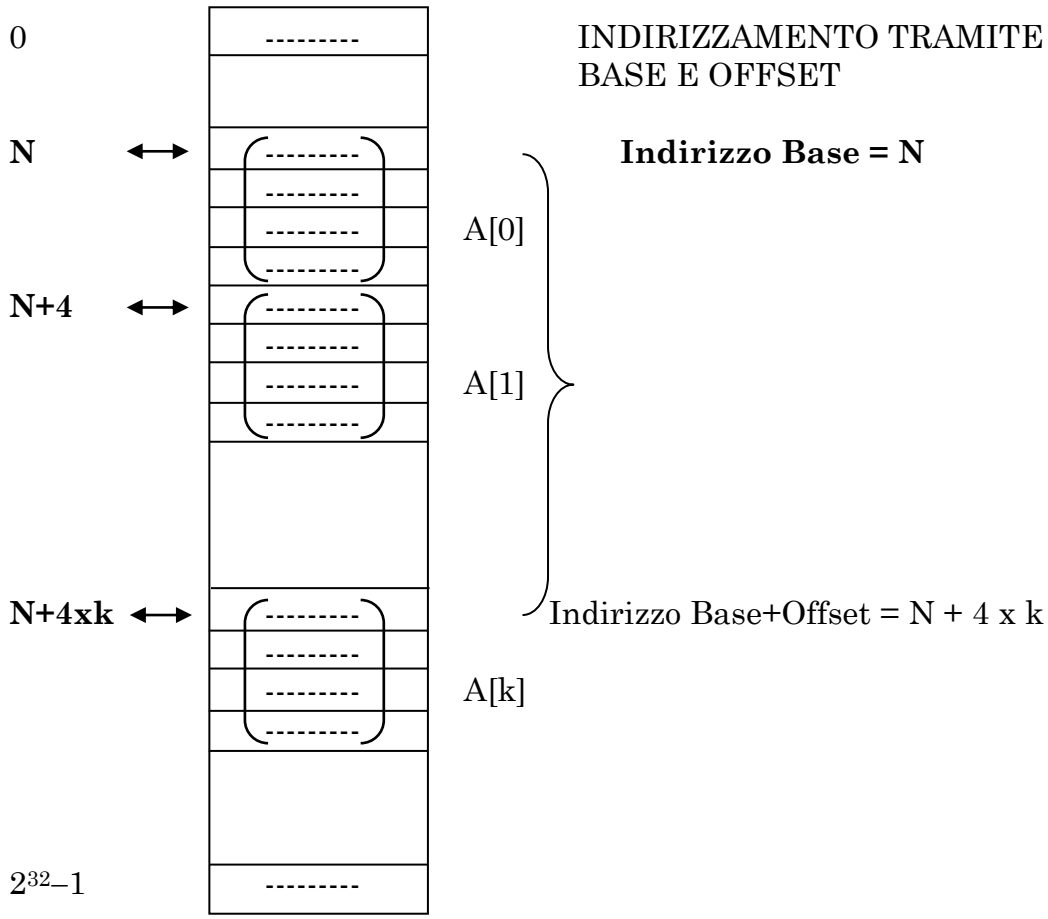
Questo porta ad assumere l'indirizzo del PRIMO BYTE del PRIMO ELEMENTO di un Array come INDIRIZZO BASE per gli accessi in Memoria utilizzando l'OFFSET per la gestione degli indici dell'Array.

L'indirizzo fornito delle istruzioni *Load* e *Store* viene calcolato dall'Hardware del computer automaticamente durante l'esecuzione dell'istruzione.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il seguente schema fornisce una rappresentazione grafica della regola per la gestione degli elementi di un Array riferita al caso di un Array di interi, in cui ogni elemento occupa 4 locazioni di Memoria:



Quando un elemento dell'Array è rappresentato mediante un **INDICE COSTANTE** la Modalità di Indirizzamento di *Load* e *Store* consente di ottenere l'indirizzo di accesso in Memoria semplicemente indicando un OPPORTUNO VALORE dell'*Offset*.

Poiché tutti gli elementi dell'Array sono dello stesso Tipo di dato, il valore dell'*Offset* è ottenuto moltiplicando il numero di Byte necessari a memorizzare un elemento per il valore dell'indice:

OFFSET = dimensione dell'elemento dell'Array x indice.

Quando, invece, l'indice dell'elemento dell'Array è dato dal valore di una **VARIABILE**, bisogna effettuare il CALCOLO DELL'INDIRIZZO COMPLETO mediante istruzioni Assembly con la regola della Modalità di Indirizzamento tramite Base e Offset:

calcolo dell'indirizzo completo

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **Calcolo del valore dell'Offset** moltiplicando per 4, per 2, per 1 il valore della variabile che rappresenta l'indice a seconda che il dato sia rappresentato da una *parola*, da *mezza parola*, da un *Byte*.

  La moltiplicazione per 4 o per 2 può essere eseguita utilizzando, rispettivamente, l'istruzione *sll* di *shift logico a sinistra* di 2 o 1 posizione.
- **Calcolo dell'indirizzo completo** addizionando il valore ottenuto dell'*Offset* con l'*indirizzo Base dell'Array*, e memorizzando il risultato in un registro di appoggio;

l'**indirizzo di accesso in Memoria** così calcolato viene utilizzato in istruzioni *Load* o *Store* ponendo il registro di appoggio che lo contiene come registro Base e indicando l'Offset uguale a 0.

ESEMPIO Array A[ ] di numeri interi, INDICE VARIABILE

val = A[k] ;

A[k] = val ;

scegliendo i registri:	Base A [ ]	val	k
	$s0	$t1	$t3

la traduzione in Assembly MIPS è:

```
sll  $t5,  $t3,  2          #  $t5  contiene l'Offset = 4 x k
add  $t6,  $t5,  $s0        #  $t6 contiene l'indirizzo completo di A[k]
lw   $t1,  0 ($t6)          #  val = A[k]
sw   $t1,  0 ($t6)          #  A[k] = val
```

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Esercizi sulla gestione degli Array in Assembly MIPS

ESERCIZIO 1

Tradurre in Assembly MIPS le seguenti istruzioni in Linguaggio C che eseguono l'azzeramento degli elementi di un Array di INTERI.

```
int  V[50],  i,  reset  ;
for ( i = 0;  i != 50  ;  i = i+1 ) {
        V[i] = 0  ;  }
reset  =  1 ;
```

Assegnando i registri:

Base V[ ]	i	reset
$s1	$t0	$s7

una traduzione in Assembly MIPS è data da:

```
add $t0, $zero, $zero          #  i = 0
        addi $s6, $zero, 50    #  $s6  contiene il valore 50
FOR     beq $t0, $s6,  END_FOR          # uscita dal ciclo for
                                # se i == 50 è vera   ⟺
                                # se i != 50 è falsa
        sll  $t2,  $t0,  2     #   $t2  contiene l'Offset = 4 x i
        add  $t3, $t2, $s1     #  $t3  contiene l'indirizzo completo
                               #  di V[i]
        sw  $zero,  0 ($t3)    #   V[i] = 0
        addi $t0, $t0,  1      #   i= i+1
        j  FOR
END_FOR addi  $s7,  $zero,  1          #   reset  =  1
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ESERCIZIO 2

Tradurre in Assembly MIPS le seguenti istruzioni in Linguaggio C che effettuano lo scambio di posizione tra due elementi di un Array di INTERI.

```
if ( A[k+1]  <  A[k] )  {
        temp = A[k]  ;
        A[k] = A[k+1]    ;
        A[k+1]  =  temp   ;
}
temp = 0   ;
```

assegnando i registri: Base A[ ] k temp

$s0 $t1 $t3.

Osserviamo che l'indirizzo dell'elemento A[k+1] si può ottenere da quello dell'elemento A[k] aggiungendo il valore 4 che è il numero di Byte necessari per memorizzare un elemento dell'Array.
Quindi nella traduzione si utilizza l'indirizzo di A[k] con l'Offset 0 per accedere all'elemento A[k] e con l'Offset 4 per accedere all'elemento A[k+1].

```
        sll  $t4,  $t1,  2          #  $t4  contiene l'Offset = 4 x i
        add  $t5, $t4, $s0          #  in $t5 l'indirizzo completo di A[k]
        lw  $s6,  0 ($t5)           # copia in $s6 l'elemento A[k]
        lw  $s7,  4 ($t5)           # copia in $s7 l'elemento A[k+1]
        slt $t0, $s7, $s6           # pone 1 in $t0 se A[k+1] < A[k]
                                            # è vera, altrimenti pone 0 in $t0
        beq $t0, $zero, END_IF      # uscita  dall'if  senza scambio
        sw $s7, 0($t5)              #  A[k] = A[k+1]
        sw $s6, 4($t5)              #  A[k+1] = A[k]
END_IF  addi  $t3,  $zero,  0       #   temp  =  0
```

Gli attuali Compilatori effettuano anche ottimizzazioni del codice tradotto.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Richiamo delle nozioni relative alla gestione delle stringhe

Il Tipo di dato stringa è una struttura dati costituita da una sequenza di caratteri e da operatori che effettuano elaborazioni come la concatenazione di stringhe ed il confronto.

Nei Linguaggi C, C++ e Java una costante di Tipo stringa è rappresentata racchiudendola tra doppi apici (ad esempio “esempio”).

Una costante di Tipo carattere è rappresentata inserendo tra singoli apici il carattere (ad esempio ‘a’) o la corrispondente sequenza di escape (ad esempio ‘\097’) costituita dalla barra rovesciata seguita dal codice ottale o esadecimale.

Anche se non è un Tipo di dato primitivo, i Linguaggi ad Alto Livello ne consentono la gestione con modalità differenti.

Il Linguaggio C implementa il Tipo di dato stringa mediante un ARRAY di caratteri ASCII, che sono rappresentati mediante 8 bit. L’ultimo elemento della stringa è il carattere NULL che ha codice 0.

Poiché lo standard ASCII assegna codice non nullo 48 al carattere 0, è possibile avere all’interno della stringa anche caratteri 0 senza possibilità di confusione con il carattere di fine stringa NULL.

Le istruzioni Assembly MIPS di trasferimento dati usate per la gestione delle stringhe di caratteri con codifica ASCII sono Load Byte unsigned (lbu) e Store Byte (sb).

Per il calcolo dell’Offset bisogna tenere conto del fatto che un elemento di un Array che implementa una stringa di caratteri ASCII è memorizzato in UNA SOLA locazione di Memoria (un Byte).

Quindi, in questo caso, l’Offset è uguale al valore dell’indice, Quando, invece, un elemento di una *stringa* è indicato mediante un indice VARIABILE, bisogna calcolare l’indirizzo completo di accesso in Memoria applicando la regola della Modalità di Indirizzamento tramite Base e Offset:

calcolo dell’indirizzo completo

- **Calcolo del valore dell’Offset:**

  Offset = dimensione dell’elemento x indice = 1 x indice = indice

  poiché il codice di un carattere ASCII occupa una locazione.

- **Calcolo dell’indirizzo completo** addizionando il valore ottenuto dell’*Offset* con l’*indirizzo Base dell’Array*.

L’**indirizzo completo** così calcolato viene utilizzato in istruzioni *Load* o *Store* ponendo il registro che lo contiene come registro Base e indicando l’Offset uguale a 0.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d’autore (L. 22.04.1941/n. 633).

Nel caso di un Array di caratteri con codifica **Unicode UTF-16** ogni elemento dell'Array occupa **DUE LOCAZIONI di Memoria**. Quindi, le istruzioni Assembly MIPS di trasferimento dati usate per la gestione delle *stringhe* di caratteri con codifica Unicode a 16 bit sono *Load half word unsigned* (lhu) e *Store half word* (sh), rispettivamente per l'accesso alla Memoria in lettura e in scrittura.

In questo caso per il calcolo dell'indirizzo dell'elemento testo[k] occorre moltiplicare per 2 l'indice k (mediante lo *shift* a sinistra di una sola posizione) per ottenere l'Offset da sommare all'indirizzo Base dell'Array.

Il Linguaggio Java utilizza la codifica Unicode a 16 bit ed implementa il Tipo di dato *stringa* con la classe S*tring*, quindi le stringhe sono oggetti e gli operatori sono metodi della classe.

Il Linguaggio C++ supporta entrambe le implementazioni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Esercizi sulla gestione delle stringhe in Assembly MIPS

ESERCIZIO 3

La variabile cont fornisce il numero di spazi bianchi nella *stringa* testo[ ] e la variabile lun la lunghezza della *stringa*.

```
        cont_a = 0  ;
        k = 0   ;
        while  ( testo[k] != 0 ) {
                if ( testo[k] == '\032' )
                        cont = cont +1   ;
                k = k+1 ;
        }
        lun = k ;
```

assegnando:	Base testo[ ]	cont	lun	k
	$s3	$s1	$s2	**$t7**

una traduzione in Assembly MIPS è data da:

```
                add $s1, $zero, $zero      #  cont = 0
                add $t7, $zero, $zero      #  k = 0
                ori $t6, $zero, 32         # pone in $t6 il codice 32
WHILE           add  $t2, $t7, $s3         #  $t2  contiene l'indirizzo completo
                                           #  di  testo[k]
                lbu  $t1,  0 ($t2)         # copia in $t1 il carattere testo[k]
                beq $t1, $zero, END_WHILE          # uscita dal ciclo while
                                                   # se testo[k] è uguale a NULL
                bne $t1, $t6, END_IF       # uscita dall'if  se testo[k] è
        # diverso da spazio bianco

                addi $s1, $s1,  1      #   cont = cont +1
END_IF          addi $t7, $t7,  1      #   k = k+1
                j  WHILE
END_WHILE   add  $s2,  $zero,  $t7     #  lun = k
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

OSSERVAZIONI

L'uso dell'istruzione *bne* che richiede operandi rappresentati con 32 bit è possibile poiché i codici ASCII di 8 bit di entrambi i caratteri scritti in $t1 e $t6 sono completati automaticamente a 32 bit dalle istruzioni *ori* e *lbu*.

Il carattere costante spazio bianco (codice ASCII decimale 32) viene scritto nel registro $t6 per essere confrontato ad ogni iterazione con l'elemento della *stringa* letto in Memoria. Anche se il carattere figura all'interno del ciclo *while,* si ottimizza la traduzione effettuando una sola scrittura nel registro $t6 fuori dal ciclo *while*. Gli attuali Compilatori effettuano anche ottimizzazioni del codice tradotto.

La condizione di arresto del *while* si incontra sicuramente, perché si presenta quando il carattere letto è uguale al simbolo NULL che è sempre presente come simbolo di fine *stringa*.

Ricordiamo che l'inizializzazione di una *stringa* mediante i doppi apici nella dichiarazione di Tipo associa un Array di dimensione pari alla lunghezza della *stringa* (numero di caratteri) più 1, in quanto i doppi apici aggiungono automaticamente il carattere NULL di fine *stringa*. Quindi, una dichiarazione come la seguente:

char testo[7] = "esempio" ;

determina un errore in fase di compilazione perché la dimensione dell'Array è minore dei caratteri da inserire, che sono i 7 caratteri della lunghezza della parola esempio più un carattere NULL.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ESERCIZIO 4

Copia della stringa di caratteri ASCII y[ ] nella stringa x[ ].

```
char   y[140]  , x[140]   ;
int  k   ;
x[0]  =  y[0]    ;
k = 0   ;
while  ( y[k] != 0 ) {
        x[k]  =  y[k]   ;
        k = k+1   ;
}
k = 3  ;
```

Assegnando:	Base x[ ]	Base y[ ]	k
	$s1	$s2	**$t3**

una traduzione in Assembly MIPS è data da:

```
                add $t3, $zero, $zero   #  k = 0
WHILE           add  $t6, $t3, $s2      # $t6 contiene l'indirizzo completo
                                        #  di  y[k]
                lbu  $t0,  0 ($t6)      #  copia in $t0 il carattere  y[k]
                add  $t5, $t3, $s1      # $t5 contiene l'indirizzo completo
                                        #  di  x[k]
                sb  $t0,  0 ($t5)       #    x[k]  =  y[k]
                beq $t0, $zero, END_WHILE       # uscita dal ciclo while
                                        # se y[k] è uguale a NULL
                addi $t3, $t3,  1       #   k = k+1
                j  WHILE
END_WHILE   addi  $t3,  $zero,  3     #  k = 3
```

Osserviamo che l'assegnamento x[0] = y[0] deve essere fatto prima del test di uscita per poter copiare in x[ ] anche la stringa y[ ] vuota.

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- D. A. Patterson, J. L. Hennessy, "Struttura e progetto dei calcolatori", Zanichelli.
- S.B. Lippman, J. Lajolie, C++ corso di programmazione, Addison Wesley.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).